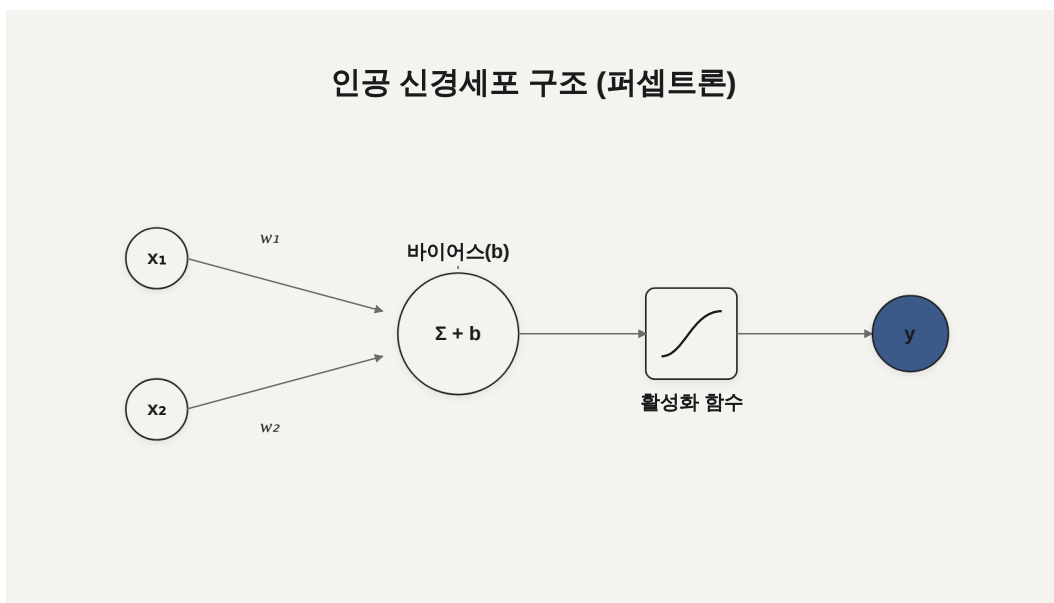


STEP 1. ANN/DNN 기초 + 역전파 (1주)

1. 인공 신경망의 기본 원리 이해하기

인공 신경망은 인간의 뇌를 모방하여 만든 수학적 모델로, 복잡한 문제를 해결하는 데 사용됩니다.



1.1. 인공 신경망의 구성 요소

인공 신경망은 여러 개의 뉴런(퍼셉트론)이 연결되어 정보를 처리합니다.

- **인공 뉴런 (퍼셉트론):** 입력값, 가중치, 편향을 이용해 출력을 내는 수학적 모델입니다.
 - 입력값: 신경망으로 들어오는 데이터입니다.
 - 가중치: 각 입력값의 중요도를 나타내는 값으로, 중요한 입력일수록 높은 가중치를 가집니다.
 - 예시: 학생의 중간고사 점수를 예측할 때, 학업 성취도가 출석률보다 더 높은 가중치를 가집니다.
 - 편향: 활성화 함수의 경계를 조절하는 값입니다.

- 뇌세포의 역치처럼, 일정 수준 이상의 자극이 와야 다음 뉴런으로 정보를 전달하는 것과 유사합니다.
- 활성화 함수: 뉴런의 최종 출력을 결정하고, 다음 뉴런으로 정보를 전달할지 말지를 정하는 함수입니다.
 - 이를 통해 신경망에 층을 쌓는 효과를 줄 수 있습니다.
- 가중합: 활성화 함수를 거치기 전, 입력값에 가중치와 편향이 적용된 결과입니다.
 - 이는 직선의 방정식으로 표현될 수 있으며, 데이터 분류의 결정 경계 역할을 합니다.

1.2. 퍼셉트론의 작동 방식

퍼셉트론은 입력값에 가중치를 곱하고 편향을 더한 후, 활성화 함수를 통과시켜 최종 출력을 계산합니다.

- 수식으로 표현하면 다음과 같습니다:
$$y = f(x_1 \cdot w_1 + x_2 \cdot w_2 + b)$$

1.3. 활성화 함수의 역할

활성화 함수는 뉴런의 출력을 결정하며, 신경망에 층을 쌓는 효과를 부여합니다.

- **시그모이드 함수**: 많이 사용되는 활성화 함수 중 하나로, 입력값이 0보다 크면 0.5보다 큰 값을, 작으면 0.5보다 작은 값을 출력합니다.
 - 이를 통해 특정 임계값(예: 0.5)을 기준으로 다음 뉴런으로 정보를 전달할지 여부를 결정할 수 있습니다.

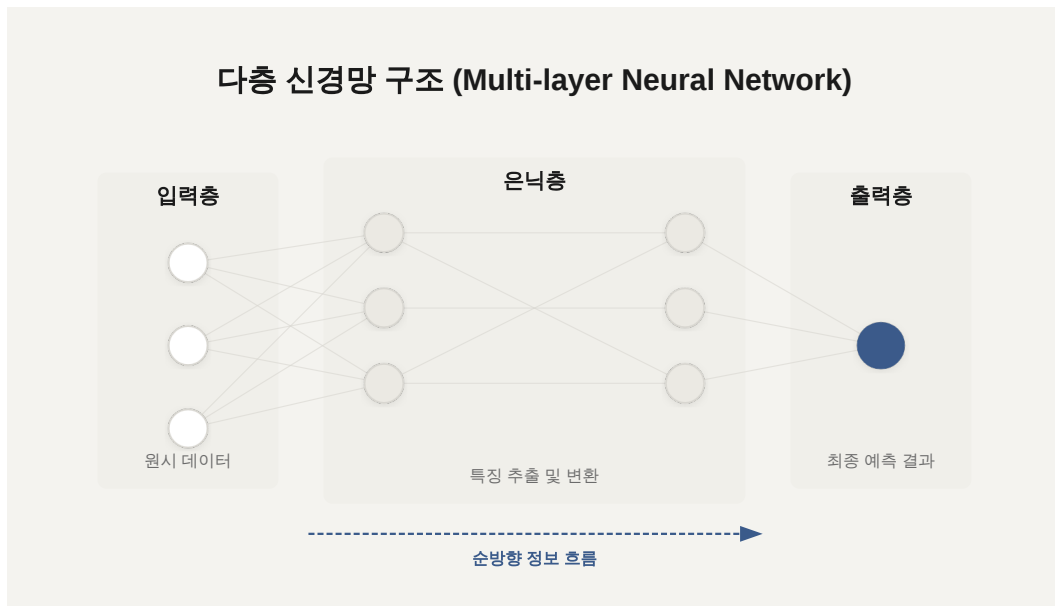
1.4. 가중합과 결정 경계

가중합은 활성화 함수를 통과하기 전의 값으로, 직선의 방정식처럼 표현될 수 있습니다.

- 이 직선은 데이터를 분류하는 '결정 경계' 역할을 합니다.
 - 직선 위에 있는 데이터는 1로, 아래에 있는 데이터는 0으로 분류하는 식입니다.
- 하지만 단일 직선으로는 모든 데이터를 분류할 수 없는 경우가 있습니다.
 - 예를 들어, XOR(배타적 논리합) 문제는 직선 하나로는 분류가 불가능합니다.

2. 다층 신경망과 학습 방법

단일 직선으로 해결할 수 없는 복잡한 데이터 분류 문제를 해결하기 위해 다층 신경망이 등장했습니다.



2.1. 다층 신경망의 아이디어

다층 신경망은 여러 개의 직선(결정 경계)을 사용하여 데이터의 분포를 변환하고, 최종적으로 분류가 가능하게 만듭니다.

- **데이터 분포 변환:** 여러 개의 층을 거치면서 데이터의 위치나 분포를 효과적으로 바꿔줍니다.
 - 예시: XOR 문제를 해결하기 위해 두 개의 선을 사용하여 데이터를 새로운 축으로 변환하면, 하나의 직선으로 분류가 가능해집니다.
- **층의 깊이와 너비:**
 - 층의 깊이는 신경망에 쌓인 층의 개수를 의미합니다.
 - 층의 너비는 각 층에 있는 뉴런(노드)의 수를 의미합니다.

2.2. 신경망 학습의 핵심: 가중치와 오차

신경망은 올바른 가중치 값을 찾아 데이터를 정확하게 분류하는 것을 목표로 합니다.

- **가중치 계산:** 신경망의 각 층에서 가중치가 계산되며, 이 과정에서 데이터의 분포가 변환됩니다.
 - 예시: 입력값 X_1 , X_2 와 가중치 W_1 , W_2 , 편향 B_1 을 이용해 은닉층의 출력 A_1 을 계산합니다.
 - 이후 계산된 은닉층의 출력을 이용해 최종 출력값을 계산합니다.
- **손실 함수:** 실제 값과 신경망이 예측한 값의 차이를 나타내는 함수입니다.
 - 오차가 적을수록 더 좋은 결정 경계를 가진 모델이라고 판단합니다.

- 점과 결정 경계 사이의 거리를 오차로 지정하여 좋은 직선을 찾는 데 활용합니다.
- **과적합 (Overfitting):** 학습 데이터에 너무 완벽하게 맞춰져서, 실제 데이터에서는 성능이 떨어지는 현상입니다.
 - 일반적으로 95~98% 정도의 정확도가 좋은 성능으로 간주됩니다.

2.3. 가중치 학습 방법: 경사 하강법

실제로는 알 수 없는 가중치 값을 찾아내기 위해 경사 하강법을 사용합니다.

- **경사 하강법의 원리:** 손실 함수의 최소값을 찾기 위해 현재 지점에서 기울기의 반대 방향으로 조금씩 이동하는 방법입니다.
 - 손실 함수 그래프에서 가장 낮은 지점을 찾는 과정과 같습니다.
 - 기울기가 가파를수록 많이 이동하고, 완만할수록 적게 이동합니다.
- **학습률:** 가중치를 얼마나 업데이트할지 결정하는 값입니다.
 - 보폭(업데이트 크기)은 학습률과 기울기의 곱으로 결정됩니다.
 - 학습률이 너무 크면 최적점을 지나칠 수 있고, 너무 작으면 학습이 느려질 수 있습니다.
 - 일반적으로 0.0001 정도의 학습률을 사용합니다.
- **최적화점의 한계:** 경사 하강법은 시작점에 따라 다른 최소값을 찾을 수 있으며, 반드시 전역 최소값(가장 작은 값)을 보장하지는 않습니다.

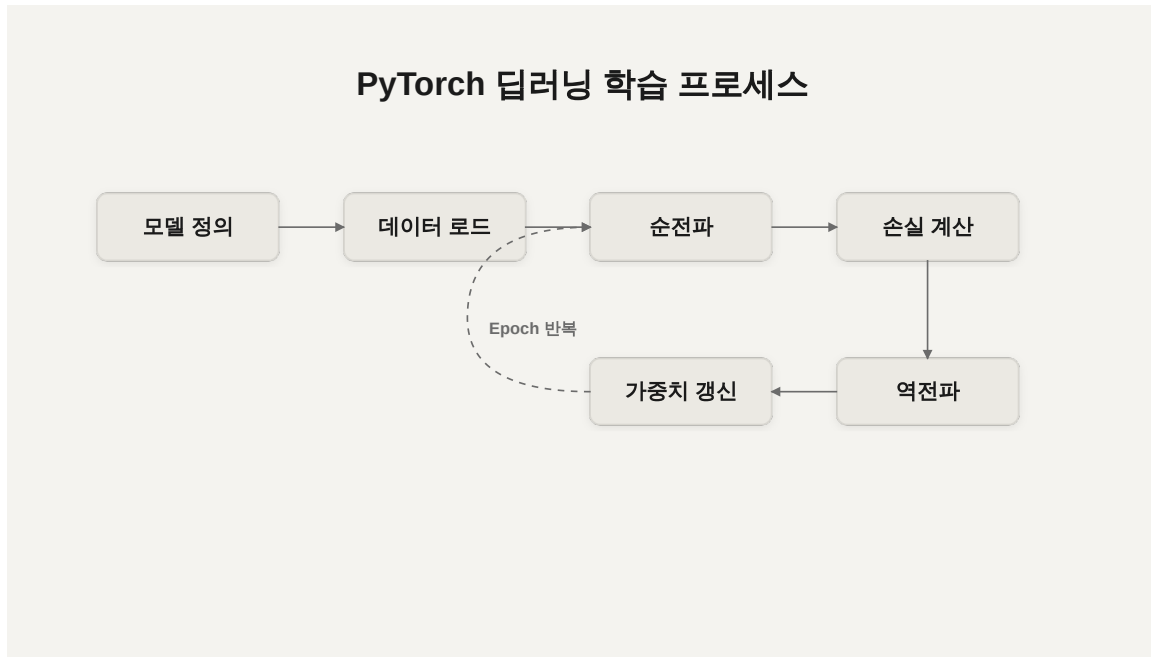
2.4. 오차 역전파 알고리즘

가중치가 많아 직접 미분하기 어려울 때, 오차 역전파 알고리즘을 사용하여 효율적으로 가중치를 업데이트합니다.

- **연쇄 법칙 활용:** 복잡한 함수의 미분을 중간값들을 이용하여 나누어 계산합니다.
 - 어려운 미분 $d(y)/d(x)$ 를 쉬운 미분 $d(y)/d(f)$ 와 $d(f)/d(x)$ 로 나누어 계산하고 곱합니다.
- **역방향 계산:** 입력에서 출력으로 가는 순전파와 달리, 출력층에서부터 입력층 방향으로 거슬러 올라가며 가중치를 수정합니다.
- **핵심:** 정답과 예측값의 오차를 최소화하는 가중치를 찾는 알고리즘으로, 연쇄 법칙을 이용해 출력층부터 가중치를 수정해 나갑니다.

3. 파이토치를 이용한 신경망 구현

파이토치 라이브러리를 사용하여 실제 신경망을 만들고 학습하는 과정을 살펴봅니다.



3.1. 사인 함수 예측 신경망 만들기

사인 함수 값을 예측하는 간단한 신경망을 만들어봅니다.

- **학습 과정:**

1. 모델 정의
2. 순전파 (모델 예측값 출력)
3. 오차 계산
4. 오차 역전파 (가중치 수정)
5. 반복 학습 (원하는 만큼)

- **데이터 생성:** 사인 함수 예측을 위해 파이 부터 파이 까지의 구간을 천 개의 점으로 나누어 데이터를 직접 생성합니다.

- **모델 구성:** 사인 함수는 3차 다항식으로 근사 가능하므로, 3차 다항식의 계수(a, b, c, d)를 가중치로 학습합니다.

- **학습 루프:**

- 학습률(1e-6)과 반복 횟수(2000 에포크)를 지정합니다.
- 예측값과 실제값의 평균 제곱 오차(MSE)를 손실 함수로 사용합니다.
- 기울기를 계산하고 경사 하강법에 따라 가중치를 업데이트합니다.

- **결과:** 학습 후 예측값은 실제 사인 함수와 유사한 그래프를 보이지만, 랜덤한 가중치로 생성된 그래프는 완전히 다릅니다.

3.2. 보스턴 집값 예측 신경망 만들기

실제 데이터를 사용하여 집값을 예측하는 신경망을 만듭니다.

- **데이터 로딩:** Pandas와 Scikit-learn 라이브러리를 사용하여 보스턴 집값 데이터를 불러옵니다.
 - 데이터셋은 범죄율, 방 개수 등 다양한 특징(feature)과 집값(target)으로 구성됩니다.
- **모델 정의:**
 - PyTorch의 `torch.nn` 모듈을 사용하여 MLP(다층 퍼셉트론) 모델을 구성합니다.
 - `Sequential` 을 이용해 여러 개의 신경망 층(Linear, ReLU)을 순서대로 쌓습니다.
 - `Linear` : 입력과 출력의 개수를 지정하는 신경망 층입니다.
 - `ReLU` : 0보다 작은 값은 0으로, 그 이상은 자기 자신을 출력하는 활성화 함수입니다.
 - 마지막 층은 예측하고자 하는 집값(하나의 값)을 출력합니다.
- **학습 루프:**
 - 배치 크기(100개씩)와 학습률(0.001)을 설정합니다.
 - 데이터를 PyTorch 텐서 형태로 변환하고, 최적화 기울기를 초기화합니다.
 - 평균 제곱 오차(MSE)를 손실 함수로 사용하고, 오차 역전파를 통해 가중치를 업데이트합니다.
- **성능 평가:** 예측값과 실제 집값의 차이가 크지 않음을 확인합니다.

3.3. 손글씨 분류 신경망 만들기

이미지 데이터를 입력받아 0부터 9까지의 숫자를 분류하는 신경망을 만듭니다.

- **회귀 vs 분류:**
 - 회귀: 값을 직접 예측하는 문제 (예: 집값 예측).
 - 분류: 어떤 범주에 해당하는지 예측하는 문제 (예: 손글씨 분류).
- **데이터 로딩:** PyTorch의 `torchvision.datasets.MNIST` 를 사용하여 손글씨 이미지 데이터셋을 불러옵니다.

- `ToTensor` 변환을 통해 이미지를 PyTorch 텐서 형태로 변환합니다.
- **DataLoader:** 데이터를 배치 단위로 효율적으로 불러오기 위해 사용합니다.
 - 학습 시에는 데이터를 섞어주는 `shuffle=True` 옵션을 사용합니다.
- **모델 구성:**
 - 입력 이미지를 벡터 형태로 변환(784 픽셀)하고, 여러 개의 은닉층을 거쳐 최종적으로 10개의 숫자 범주 중 하나를 예측합니다.
 - GPU 사용 가능 여부에 따라 학습 장치를 지정합니다.
- **학습 루프:**
 - 학습률($1e-3$)과 반복 횟수(20 에포크)를 설정합니다.
 - 크로스 엔트로피 오차(Cross Entropy Loss)를 손실 함수로 사용합니다.
 - 이는 실제 확률 분포와 예측 확률 분포 간의 차이를 계산합니다.
 - 오차 역전파를 통해 가중치를 업데이트하고, 학습된 가중치는 파일로 저장합니다.
- **성능 평가:** 약 97%의 분류 정확도를 달성하며, 이는 이상적인 성능 범위에 해당합니다.